《计算机组成原理》实验报告

年级、专业、班级	2023 级计算机科学与技术卓越 01 班，卓越 02 班	姓名	文艺、吴怡、段雯菀

实验题目	实验三　简易单周期 CPU 实验		
实验时间	2025 年 4 月 20 日	实验地点	DS1410
实验成绩	优秀	实验性质	☐验证性 ☑设计性 ☐综合性

教师评价：

☑算法/实验过程正确；　☑源程序/实验内容提交　☑程序结构/实验步骤合理；

☑实验结果正确；　☑语法、语义正确；　☑报告规范；

其他：

评价教师签名：钟将

一、实验目的

（1）掌握不同类型指令在数据通路中的执行路径。
（2）掌握 Vivado 仿真方式。

二、实验项目内容

阅读实验原理实现以下模块：
(1) Datapath，其中主要包含 alu(实验一已完成)，PC(实验二已完成)，adder、mux2、signext、sl2(其中 adder、mux2 数字逻辑课程已实现，signext、sl2 参见实验原理)，
(2) Controller(实验二已完成)，其中包含两部分，分别为 main_decoder，alu_decoder。
(3) 指令存储器 inst_mem(Single Port Ram)，数据存储器 data_mem(Single Port Ram)；使用 Block Memory Generator IP 构造指令，注意考虑 PC 地址位数统一。(参考实验二)
(4) 参照实验原理，将上述模块依指令执行顺序连接。实验给出 top 文件，需兼容 top 文件端口设定。
(5) 实验给出仿真程序，最终以仿真输出结果判断是否成功实现要求指令。

报告创建时间：

三、实验过程或算法（源程序）

实验步骤：

1. 从实验一中，导入 alu 模块；
2. 从实验二中导入 PC、Controller 模块；
3. 从数字逻辑实验中导入多路选择器、加法器模块；
4. 使用 Block Memory，其中 inst_mem 导入 coe 文件;
5. 参考实验原理，连接各模块；
6. 导入顶层文件及仿真文件，运行仿真；

datapath 接口定义：

信号名	方向	位宽	功能描述
clk	input	1-bit	时钟信号
rst	input	1-bit	复位信号
memtoreg	input	1-bit	写入数据存储器的数据的选择信号：ALU 的计算结果 Or 存储器读取的结果
pcsrc	input	1-bit	下一条语句的地址的选择信号：PC+4or 跳转的新地址
alusrc	input	1-bit	ALU 的第二个操作数的选择信号：立即数的 32 位拓展 Or 寄存器堆读取的值
regdst	input	1-bit	写入寄存器的地址的选择信号：rt or rd
regwrite	input	1-bit	寄存器堆的写使能信号
jump	input	1-bit	是否为 jump 指令
alucontrol	input	3-bits	ALU 的控制信号
instr	input	32-bits	一条完整的指令
readdata	input	32-bits	从数据存储器中读取到的数据
zero	output	1-bit	ALU 的运算结果是否为 0 的判断信号
fpc	output	32-bits	程序计数器,指令的地址
writedata	output	32-bits	写入寄存器的值
aluout	output	32-bits	ALU 的运算结果

数据通路：

MIPS 架构 CPU 的传统流程可分为取指、译码、执行、访存、回写 (Instruction Fetch, Decode, Execution, Memory Request, Write Back) 五阶段。数据通路的作用就是按照这些流程，将器件连接起来，让数据和控制信号正确到达指定地点，并发挥相应的作用。

```
`timescale 1ns / 1ps
/*数据通路模块：将 Controller 解码的控制信号传入 datapath（Controller 输出的 memwrite 传给 Data memory，其他
的均传入 datapath），并接收 Data memory 输入的 data 和 Instruction memory 输入的 IR，输出 ALU 计算结果、写回数
据、pc 信号以及 zero 信号
    datapath 模块包含许多模块：主要可以分为 PC 和 Data 两个部分：
        PC 部分：PC->PC+4->判断 branch(IR[15:0]<<2+PC+4)->判断 jump({PCPlus4[31:28],IR[25:0]<<2})->更新 PC
        Data 部分：IR-寄存器堆->RD1、RD2(与 SignImm 选择)-ALU->ALUReult、WriteData-DataMem->ReadData、ALUResult->Result
*/

module datapath(
    input clk,rst,
    input[2:0] aluctrl,
    input memtoreg,alusrc,regdst,regwrite,pcsrc,jump,
//pcsrc=branch&zero，从外部传入
    input[31:0] readdata,                    //lw 指令从 Data memory 读出的指令
    input[31:0] IR,
    output[31:0] aluresult,
    output[31:0] writedata,
    output[31:0] pc_out,
    output zero
    );
    //PC 部分
    wire[31:0] PC_1,PC_2,PC,PCPlus4;//PC_1 为判断 branch 后的地址信号，PC_2 为判断 jump 后的地址信号
    assign pc_out=PC;
    pc gate1(clk,rst,PC_2,PC);              //PC 模块
    adder gate2(PC,32'h00000004,PCPlus4);       //PC+4

    wire[31:0] SignImm;
    wire[15:0] tmp;
```

```
    assign tmp=IR[15:0];
    signext gate3(tmp,SignImm);      //立即数符号扩展,lw、sw、addi、beq 指令需要对 IR[15:0]进行扩展

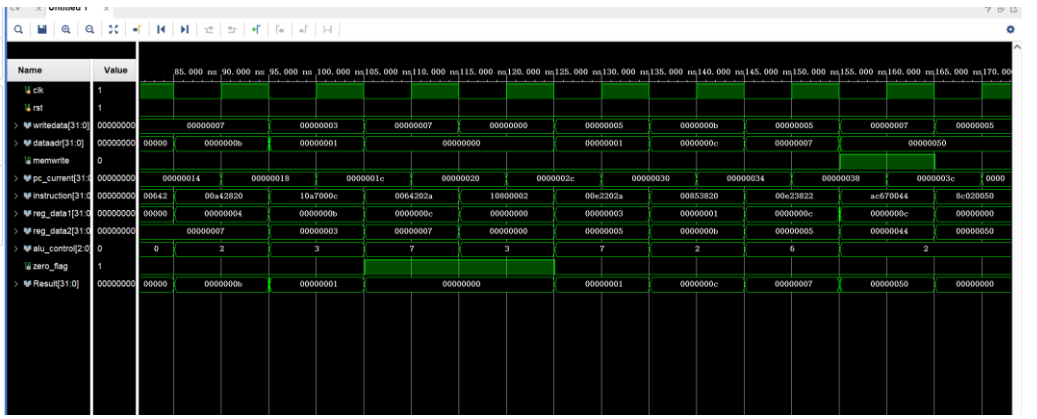
    wire[31:0] PCBranch_in;
    wire[31:0] PCBranch_out;
    sl2 gate4(SignImm,PCBranch_in);
    adder  gate5(PCBranch_in,PCPlus4,PCBranch_out);          //计算 branch 跳转指令
    mux2  gate6(pcsrc,PCPlus4,PCBranch_out,PC_1);            //判断是否执行 branch
    mux2 gate7(jump,PC_1,{PCPlus4[31:28],IR[25:0],2'b00},PC_2);      //判断是否执行 jump
    //Data 部分
    wire[4:0] WriteReg;                          //写寄存器号
    wire[31:0] Result;                           //写回寄存器数据
    wire[31:0] RD1,RD2;                          //寄存器堆读出数据
    assign writedata=RD2;
    mux2  #(5)  gate8(regdst,IR[20:16],IR[15:11],WriteReg);      //判断写寄存器号
    regfile
gate9(clk,regwrite,IR[25:21],IR[20:16],WriteReg,Result,RD1,RD2);

    wire[31:0] SrcB,ALUResult;
    assign aluresult=ALUResult;
    mux2  gate10(alusrc,RD2,SignImm,SrcB);                   //判断 RD2 or SignImm
    alu gate11(aluctrl,RD1,SrcB,ALUResult,zero);             //ALU 运算

    mux2  gate12(memtoreg,ALUResult,readdata,Result);        //判断写回寄存器堆的是 ALU 的计算结果  or lw 读取的 data

endmodule
```

四、实验结果及分析和（或）源程序调试过程



调试过程：

问题描述：
仿真 Load/Store 指令（如 lw、sw）时，data_mem 模块始终输出 Z 状态。

解决方案：
检查发现顶层 `mips` 模块未将数据存储器输出接入 `top.v` 的输出口，且读写使能信号未统一。修改 `data_mem` 实例的读写信号控制逻辑，并连接输出端口，仿真输出恢复正常。

问题描述：
仿真中观察到指令执行顺序混乱，有时跳过某些指令，PC 跳转似乎不是按 4 字节对齐。

解决方案：
确认后发现 adder 模块中 PC+4 设计有误，输入未统一为 32 位，导致地址拼接出错。修改 adder 模块，确保参与运算的 PC 和立即数为 32 位，且 jump 地址处理采用 `{PC[31:28], instr_index, 2'b00}` 拼接形式。修正后执行顺序正确。

备注：